

生成式软件开发(5)
- C/S 应用及基于Plan模式开发

大纲

01 **什么是C/S架构**

从单机应用到分布式系统的范式转变，理解客户端、服务端与数据库的角色分工

02 **客户端与服务端的通信**

深入理解HTTP协议、RESTful API设计规范与JSON数据交换格式

03 **Agentic IDE开发**

Plan模式让AI帮你规范开发流程，从架构设计到部署的完整实践

04 **Demo演示**

实际案例演示

05 **注意事项（避坑指南）**

新手开发C/S应用常见问题与解决方案：部署脚本、CORS跨域、错误处理等关键要点

核心目标：使用Plan模式开发C/S架构应用

01

什么是C/S架构

从单机应用到分布式系统的范式转变



客户端



服务端



数据库

单机应用的局限性

典型案例

- 记事本 单
- 机游戏
- Excel / Word

景点浏览页面案例

回顾 Lab 2 的景点浏览页面，这是一个典型的单机应用场景

核心问题

- 数据存储在本地 `spots.json` 文件
- 数据变化需要重新部署整个页面
- 只能在自己的机器上查看应用程序



初学者误区：误以为部署到本地服务器后所有人都能访问，实际上只有自己能访问

需要联网运行的应用

事实上，**现在绝大多数的应用都需要联网运行**

多用户需要共享数据

多个用户需要访问和修改同一份数据，如协作编辑文档、社交媒体

数据需要在不同设备间同步

在手机、平板、电脑上保持数据一致，如云存储服务

需要集中管理权限和安全

统一的用户认证、权限控制、数据加密等安全策略

需要处理复杂计算

客户端性能不足，需要服务端进行大数据处理、AI推理等

需要持续更新业务逻辑

无需重新安装客户端，服务端更新后所有用户立即生效，如在线游戏、SaaS应用

核心洞察： 当你遇到以上任何需求时，只能采用 C/S 架构

单机应用 vs 联网应用对比

对比维度	单机应用	联网应用（C/S架构）
数据存储	本地文件（JSON、SQLite等）	远程数据库（MySQL、MongoDB等）
访问范围	仅限本机访问	网络上任意设备访问
部署方式	安装到本地，更新需重新安装	服务端更新，客户端即时生效
用户协作	不支持多用户协作	支持实时多人协作
数据安全	依赖本地设备安全	服务端统一安全管理
扩展性	受限于单机性能	服务端可扩展

单机应用适用场景

个人工具、离线游戏、本地文档处理

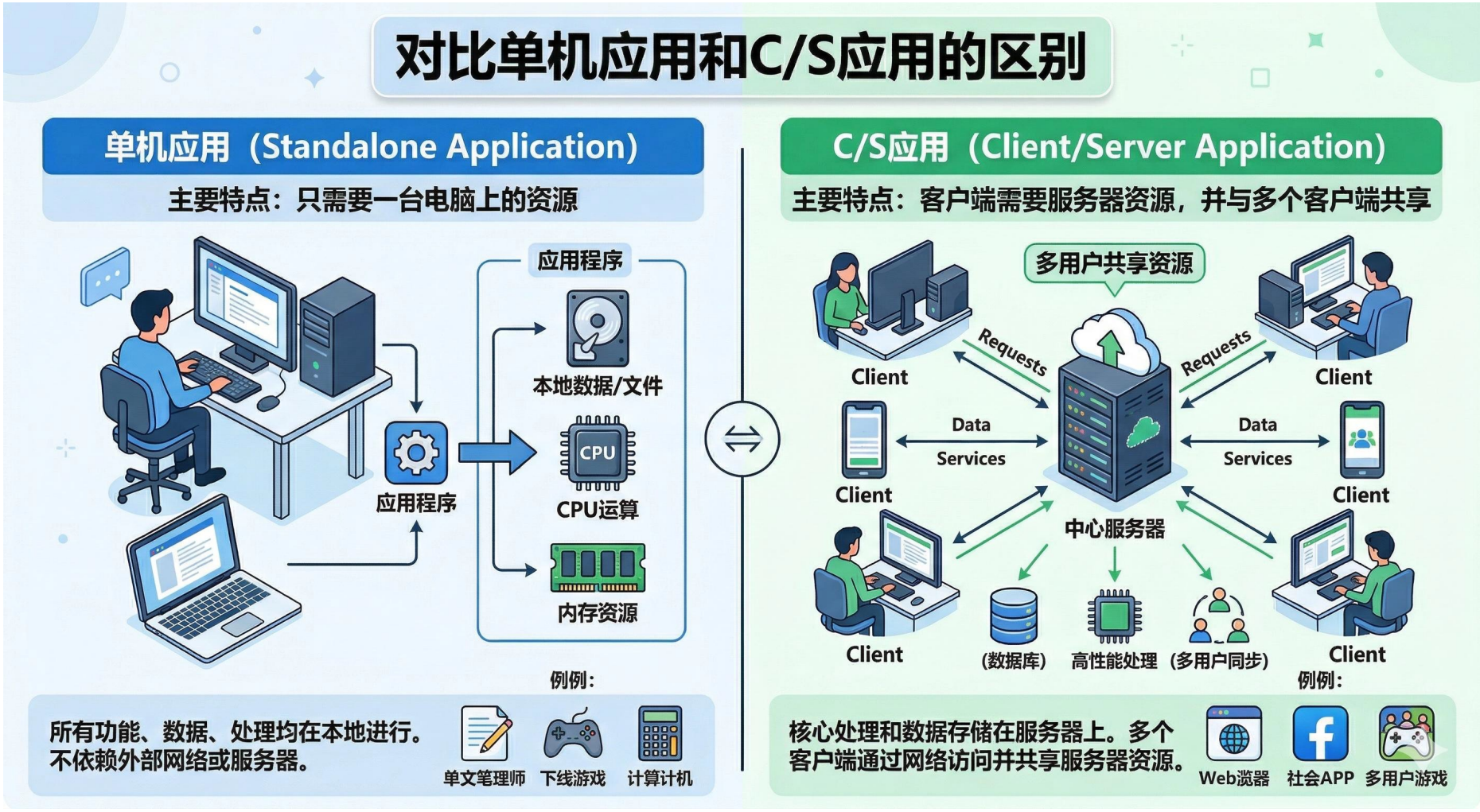
联网应用适用场景

社交平台、电商系统、协作工具、在线服务

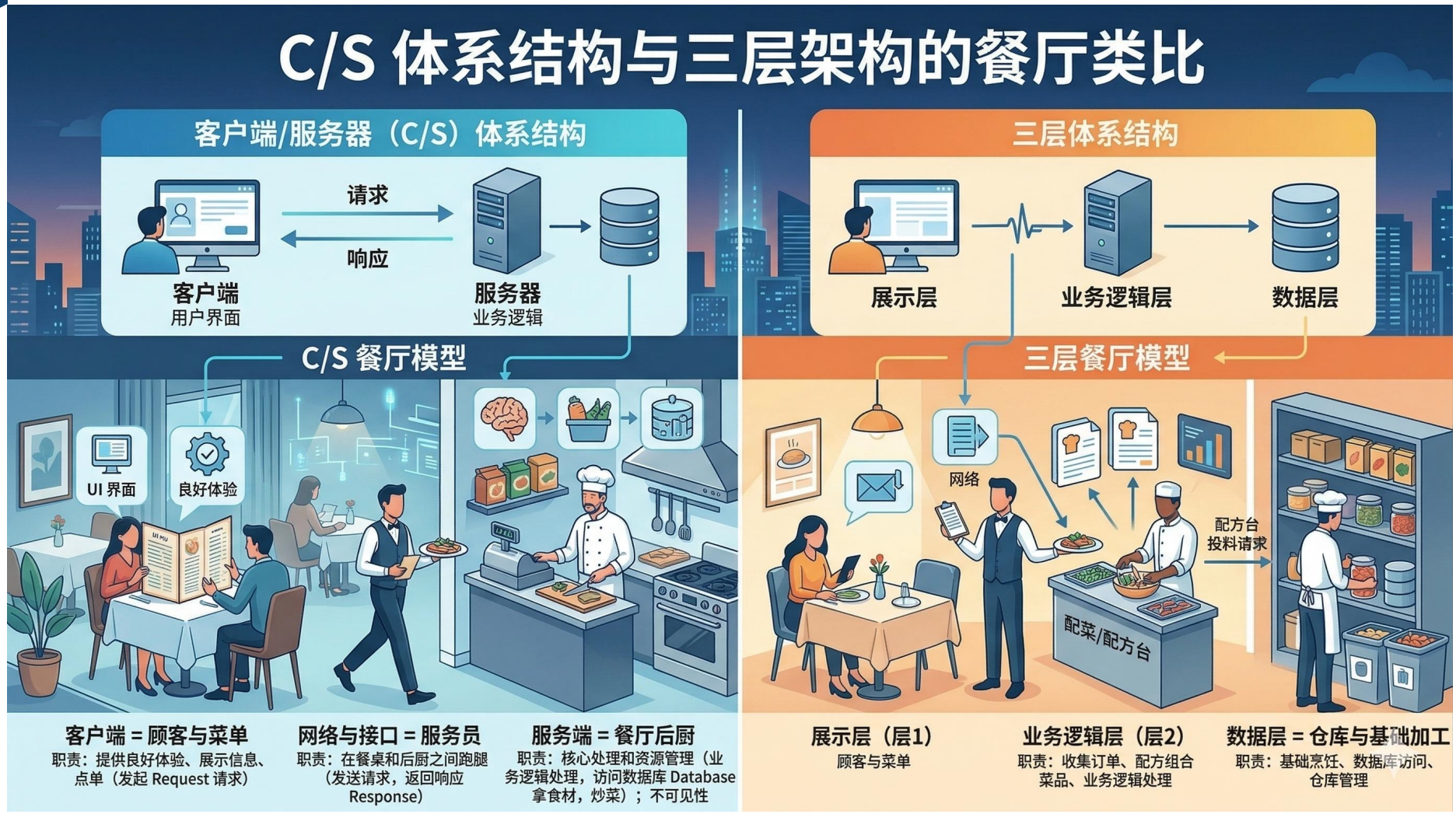
架构选择原则

根据应用需求、用户规模、数据安全要求选择

单机应用 vs 联网应用对比



餐厅模型：理解 C/S 架构


C/S 体系结构与三层架构的餐厅类比
客户端/服务器（C/S）体系结构
请求
响应
客户端
用户界面
服务器
业务逻辑
三层体系结构
展示层
业务逻辑层
数据层
C/S 餐厅模型
三层餐厅模型
UI 界面
良好体验
网络
配方台
投料请求
配菜/配方台
客户端 = 顾客与菜单
职责：提供良好体验、展示信息、点单（发起 Request 请求）
网络与接口 = 服务员
职责：在餐桌和后厨之间跑腿（发送请求，返回响应 Response）
服务端 = 餐厅后厨
职责：核心处理和资源管理（业务逻辑处理，访问数据库 Database 拿食材，炒菜）；不可见性
展示层（层1）
顾客与菜单
业务逻辑层（层2）
职责：收集订单、配方组合菜品、业务逻辑处理
数据层 = 仓库与基础加工
职责：基础烹饪、数据库访问、仓库管理

餐厅模型：理解 C/S 架构

客户端（Client）

= 顾客与菜单

顾客看着装潢漂亮的菜单（UI 界面），决定自己想吃什么并负责"点单"。客户端负责用户交互和发起请求。

服务端（Server）

= 餐厅后厨

后厨的厨师收到订单后，去仓库拿食材，然后按照菜谱炒菜。服务端负责核心的处理和资源管理。

网络与接口（Network & API）

= 服务员

服务员负责在餐桌和后厨之间跑腿，传递订单和端菜。网络负责数据传输，API定义通信规则。

工作流程

① 顾客翻看桌上的菜单（可用功能/API文档）

② 决定后告诉服务员（客户端发送 HTTP 请求）

③ 服务员把点单送到后厨（网络传输请求）

④ 后厨按订单做菜，包括取食材（数据库查询）和烹饪（业务逻辑处理）

⑤ 菜做好后，服务员端回餐桌（返回 JSON 响应）

⑥ 顾客开始享用（客户端渲染展示结果）

客户端（Client）的角色与职责

什么是客户端？

客户端是**用户直接交互的界面**，是用户与系统之间的桥梁。它负责接收用户的操作指令，向服务端发送请求，并将服务端返回的数据以可视化的方式展示给用户。

常见形式

浏览器
访问网页应用

手机App
微信、淘宝等

桌面程序
VS Code、Postman

职责

展示用户界面
渲染页面、显示数据、提供交互元素

接收用户操作
捕获点击、输入、滑动等用户行为

向服务端发送请求
通过HTTP协议发送API请求

展示返回的数据
解析JSON数据并渲染到界面

技术栈示例

Web前端
HTML5 + CSS3 + JavaScript (ES6+),
React.js， vue.js

移动端
React Native、Flutter、Swift、Kotlin

桌面端
Electron、Tauri、Qt

设计原则

- 保持界面简洁直观
- 提供即时反馈
- 优雅处理错误
- 优化加载速度

服务端（Server）的角色与职责

什么是服务端?

服务端是**提供业务处理能力的后端程序或后端系统**，是C/S架构的核心。它接收来自客户端的请求，执行相应的业务逻辑，与数据库交互，并将处理结果返回给客户端。

常见技术栈

Node.js + Express 轻量高效

JavaScript运行时，适合I/O密集型应用

Python + Flask/Django 快速开发

简洁优雅，适合快速原型和数据分析

Java + Spring Boot 企业级

稳定可靠，适合大型企业级应用

核心职责

1. **接收客户端请求**
   监听特定端口，接收HTTP请求
2. **验证数据合法性**
   参数校验、身份认证、权限检查
3. **执行业务逻辑**
   处理业务规则、计算、数据转换
4. **与数据库交互**
   查询、插入、更新、删除数据
5. **返回处理结果**
   封装JSON响应，设置状态码

数据库（Database）的角色与职责

什么是数据库？

数据库是**高效、持久化存储数据的仓库**。它提供了结构化的数据存储方式，支持高效的数据查询和写入操作，并保证数据的一致性和完整性。

常见数据库类型

关系型数据库

使用表格结构存储数据，支持SQL查询

MySQL　PostgreSQL　SQLite

文档型数据库

使用JSON-like文档存储数据，灵活schema

MongoDB　Couc DB

内存型数据库

数据存储在内存中，读写速度极快

Redis　Memcached

核心职责

长期存储数据

持久化保存应用数据，确保数据不丢失

高效查询和写入

提供索引、缓存等机制优化数据访问

保证数据一致性

事务管理、并发控制、数据完整性约束

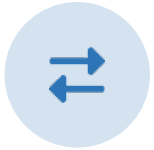
学习阶段可使用简化方案：

- ✓ 内存存储（JavaScript对象）
- ✓ JSON文件存储
- ✓ SQLite轻量级数据库

02

客户端和服务端的通信

客户端与服务端程序间通信的HTTP协议、API设计和数据交换格式 JSON



HTTP协议

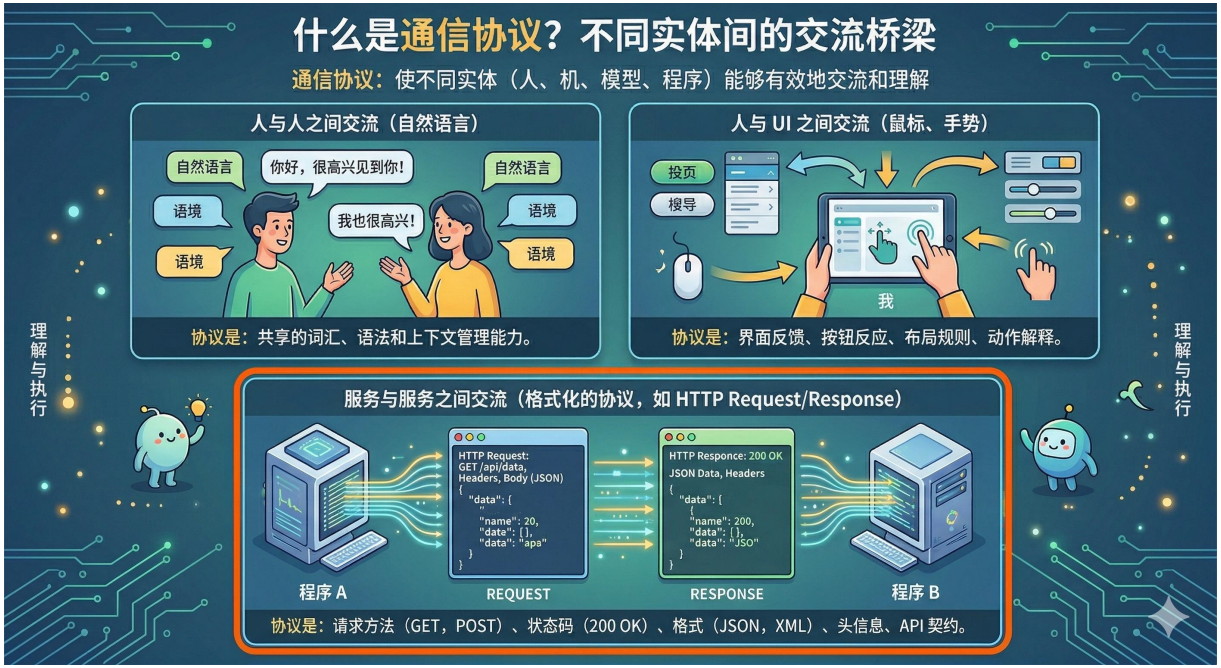


API接口



JSON格式

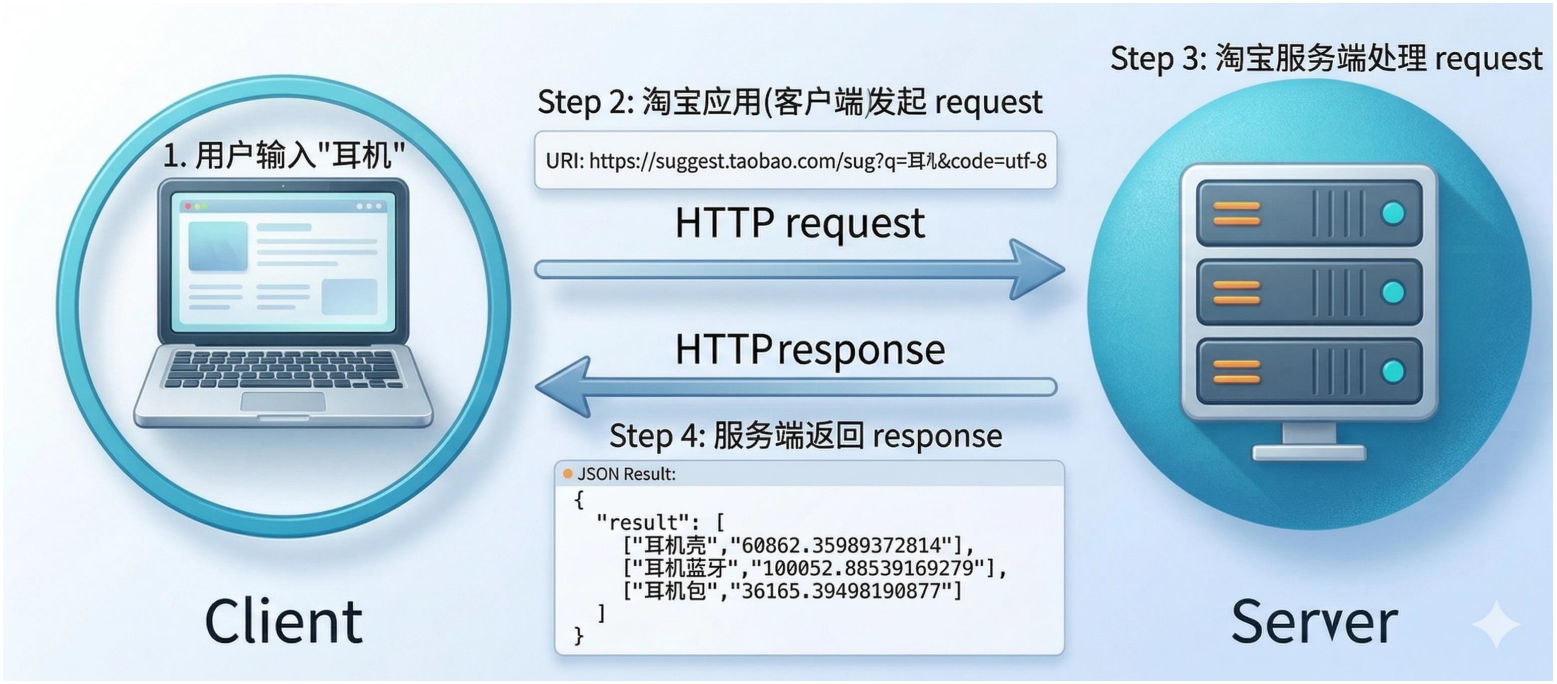
通信媒介、协议与数据格式



交互场景	通信主体	常用媒介 / 规则	特点
人与人交流	人类 ↔ 人类	自然语言（中英文等）	灵活、有歧义、富含情感
人与 UI 交互	人类 ↔ 界面	鼠标、键盘、触控	直观、可视化、即时反馈
人与大模型	人类 ↔ AI	自然语言 + Markdown	保留自然语言灵活性，Markdown 提供结构化表达
程序与程序	Client ↔ Server	HTTP 等通信协议 + JSON	**严格、精确、无歧义、机器可解析**

HTTP 请求-响应模型

为什么需要了解：C/S接口是设计文档中的重要内容



Request
客户端发送的请求
餐厅类比：顾客点菜

Response
服务端返回的响应
餐厅类比：服务员上菜

Method
请求类型（GET/POST/PUT/DELETE）
餐厅类比：动作类型（看/点/改/退菜）

URL
请求地址
餐厅类比：桌号

Status Code
状态码（200/404/500）
餐厅类比： 结果反馈

Body
请求/响应的数据内容
餐厅类比：订单详情/菜品

基于 HTTP 的通信

HTTP（超文本传输协议）是客户端和服务端之间通信的基础协议，采用请求-响应模式进行数据交换。

什么是API

API定义

API（Application Programming Interface）是**应用程序对外暴露功能的接口约定**。它定义了软件组件之间如何交互，包括可用的功能、参数格式、返回值等。

餐厅类比

菜单就是餐厅提供的API

列出所有可用功能（菜品）

顾客按菜单点菜（调用API）

遵循约定的格式和流程

厨房按菜单做菜（返回结果）

按照约定提供预期的输出

API的核心价值

模块化开发

将复杂系统拆分为独立模块，各自提供API

封装实现细节

调用方无需了解内部实现，只需知道接口

复用与集成

通过组合不同API快速构建强大应用

一句话总结

"API是现代软件开发的**乐高积木**，通过组合不同的API，可以快速构建强大的应用。"

API与函数调用的类比

对比维度	本地函数调用	远程API调用
定义方式	def get_spot(id):	GET /api/spots/{id}
调用方式	get_spot("s1")	curl /api/spots/s1
参数传递	函数参数 id	URL参数 /spots/{id}
执行位置	本地内存计算	远程服务器处理
响应时间	纳秒/微秒级	毫秒级（网络延迟）

核心洞察

API调用本质上就是**"远程函数调用"**。虽然跨越了网络边界，但概念上与本地函数调用类似，都是请求某个功能并获取结果。

关键差异

远程调用需要考虑**网络延迟、超时处理、错误重试、数据序列化**等问题，比本地调用复杂得多。

REST风格API设计约定

什么是REST?

REST（Representational State Transfer）是一种API设计风格，类似于编程语言中的命名规范，便于理解和使用。它利用HTTP协议的方法（GET/POST/PUT/DELETE）来表示对资源的操作。

RESTful API设计规范

操作	HTTP Method	URL	说明
查询列表	GET	/api/spots	获取所有景点
查询单个	GET	/api/spots/s1	获取ID为s1的景点
创建	POST	/api/spots + JSON	提交新景点数据
更新	PUT	/api/spots/s1 + JSON	修改ID为s1的景点
删除	DELETE	/api/spots/s1	删除ID为s1的景点

REST设计原则

资源导向
URL表示资源，而非动作

统一接口
使用标准HTTP方法

无状态
每个请求独立，服务端不保存状态

可缓存
响应可被客户端缓存

命名规范

- 使用小写字母
- 使用复数名词
- 使用连字符分隔
- 避免动词

什么是JSON?

JSON定义

JSON（JavaScript Object Notation）是**轻量级数据交换格式**，与JavaScript、Python中的对象/字典相似。它已成为现代Web应用数据交换的事实标准。

为什么用JSON?

人类易读
纯文本格式，结构清晰，便于理解和调试

机器易解析
标准格式，所有编程语言都有解析库

语言无关
JS/Python/Java/C++等都能处理

JSON数据结构示例

```
{
  "id": "s1",
  "name": "上海外滩",
  "city": "上海",
  "rating": 4.8,
  "price": 0.00,
  "tags": ["夜景", "历史", "免费"],
  "location": {
    "lat": 31.2304,
    "lng": 121.4737
  }
}
```

JSON数据类型

字符串 "hello"	**数字** 42, 3.14	**对象** {...}
布尔值 true, false	**数组** [1, 2, 3]	**null** 空值

调试和测试API的工具

cURL

命令行工具

cURL是一个强大的命令行工具，用于发送HTTP请求。它内置于大多数Unix/Linux系统和macOS中，是开发者和系统管理员的必备工具。

主要用途：

- **API调试：**快速测试接口是否正常工作
- **查看响应：**检查返回数据的结构和内容
- **学习理解：**直观感受HTTP请求-响应过程

cURL基本语法

```
curl [options] [URL]
```

常用选项：-X 指定方法, -H 添加头部, -d 发送数据

Postman

GUI可视化工具

Postman是一款专业的API开发协作平台，提供直观的图形界面，让API的构建、测试和文档化变得简单高效。

核心功能：

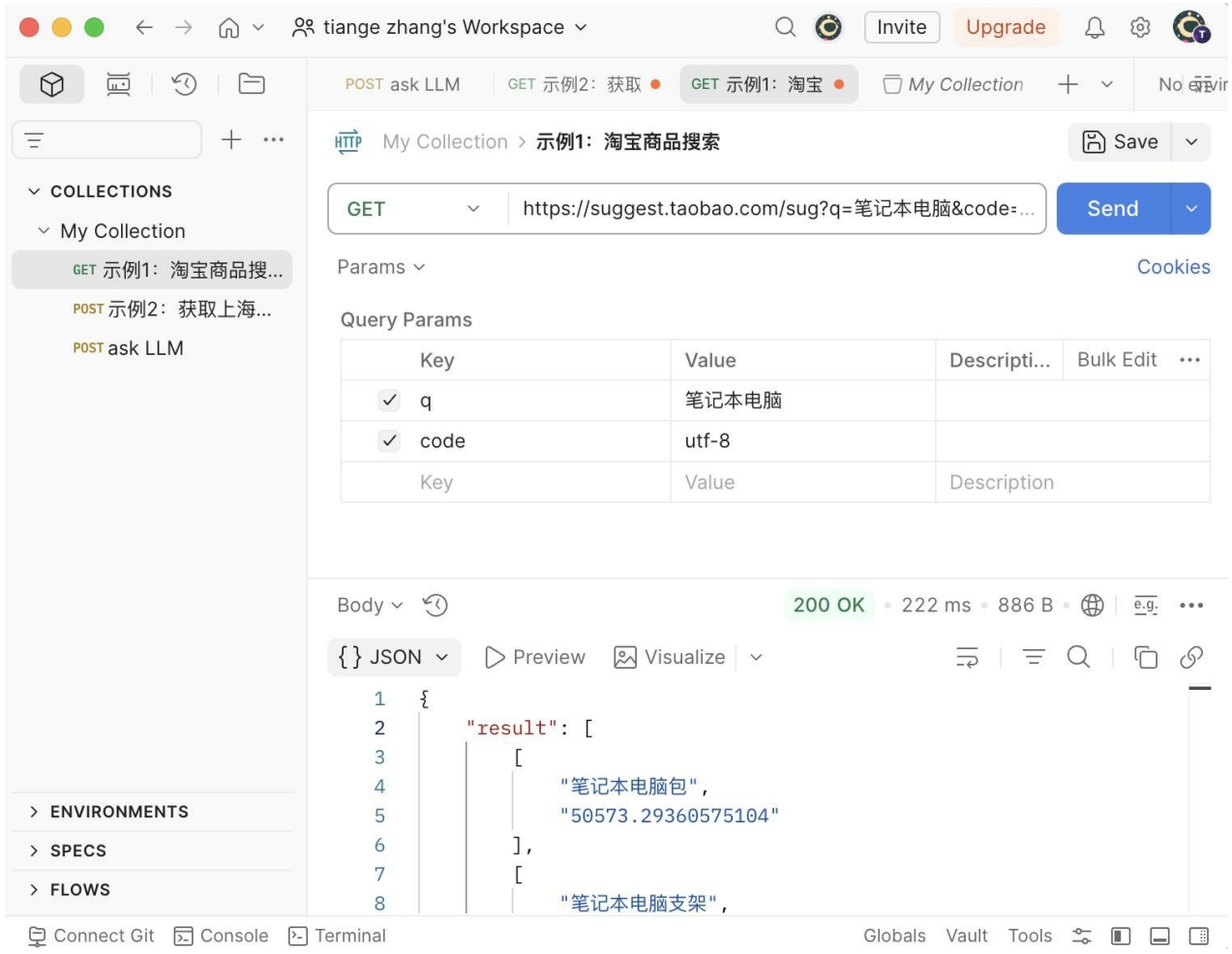
- **可视化界面：**无需记忆命令，点击即可发送请求
- **请求历史：**保存和重用之前的请求
- **环境管理：**切换开发/测试/生产环境
- **团队协作：**共享API集合和文档

工具选择建议

cURL：适合快速测试、脚本自动化、服务器环境

Postman：适合日常开发、复杂场景、团队协作

调试和测试API的工具： Postman



API示例1：淘宝商品搜索

cURL请求

```
curl "https://suggest.taobao.com/sug?q=耳机&code=utf-8"
```

请求说明： 调用淘宝搜索API，查询"耳机"商品

参数： q=耳机（搜索关键词），code=utf-8（编码格式）

核心洞察

服务端返回的是**结构化的JSON数据**，允许程序直接处理数据。前端可以解析这些数据并渲染到页面上，实现动态搜索建议功能。

- 搜索建议实时显示
- 搜索量数据可用于排序
- 支持异步加载提升体验

返回的JSON数据


```
{
  "result": [
    ["耳机", "60862.35989372814"],
    ["耳机蓝牙", "100052.88539169279"],
    ["耳机包", "36165.39498190877"],
    ["耳机头戴式", "28473.12345678901"],
    ["耳机有线", "19543.98765432109"]
  ]
}
```


数据结构解析：

- **result：** 搜索结果数组
- **每个元素：** [关键词, 搜索量]
- **搜索量：** 可用于排序和推荐

API示例2：获取天气数据

cURL请求

```
curl  "https://api.open-meteo.com/v1/forecast?latitude=31.23&longitude=121.47¤t_weather=true"
```

请求说明： 调用Open-Meteo API获取上海市天气

参数： latitude=31.23（纬度），longitude=121.47（经度），current_weather=true（当前天气）

API特点

免费开源

Open-Meteo提供免费天气API，无需注册

全球覆盖

支持全球任意经纬度的天气查询

数据丰富

提供温度、湿度、风速、降水等多种数据

返回的JSON数据


```
{
  "latitude": 31.25,
  "longitude": 121.5,
  "generationtime_ms": 0.123,
  "utc_offset_seconds": 28800,
  "timezone": "Asia/Shanghai",
  "timezone_abbreviation": "CST",
  "elevation": 18.0,
  "current_weather": {
    "temperature": 15.7,
    "windspeed": 9.6,
    "winddirection": 127,
    "weathercode": 80,
    "time": "2026-03-30T03:00"
  }
}
```


数据结构解析：

- **latitude/longitude:** 地理位置
- **temperature:** 温度(°C)
- **timezone:** 时区信息
- **windspeed:** 风速(km/h)
- **elevation:** 海拔高度
- **weathercode:** 天气代码

API示例3：访问大模型

cURL请求

```
curl -X POST "https://api.moonshot.cn/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $KIMI_API_KEY" \
  -d '{
    "model": "moonshot-v1-128k",
    "messages": [
      {"role": "user", "content": "请介绍一下上海外滩"}
    ]
  }'
```

请求说明： 调用Kimi API进行对话

方法： POST | **认证：** Bearer Token | **模型：** moonshot-v1-128k

请求结构解析

Headers（请求头）

```
Content-Type: application/json
Authorization: Bearer $KIMI_API_KEY
```

Body（请求体）

```
model: 指定使用的模型版本
messages: 对话历史数组
```

返回的JSON数据

```
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "created": 1711800000,
  "model": "moonshot-v1-128k",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "上海外滩位于上海市中心..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 15,
    "completion_tokens": 328,
    "total_tokens": 343
  }
}
```

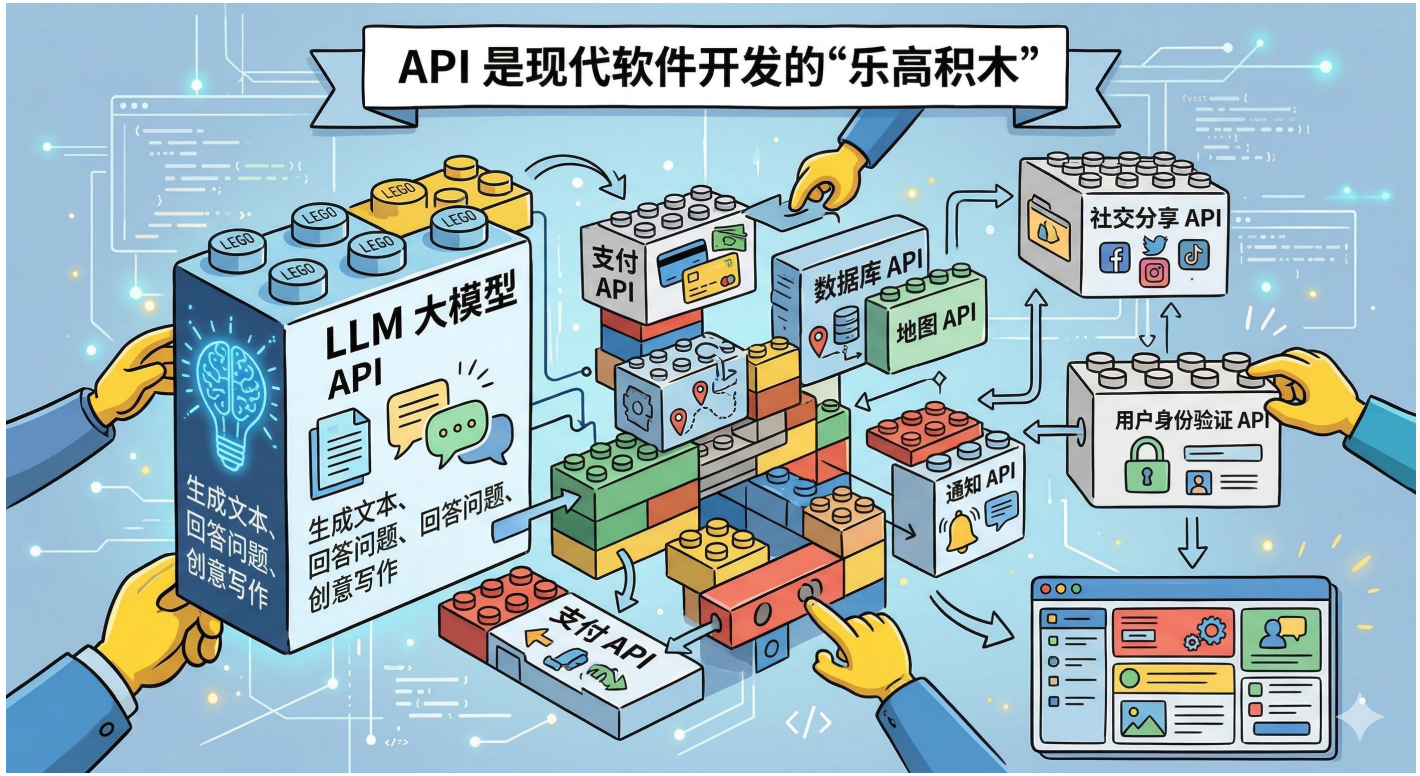
数据结构解析：

- **id：** 请求唯一标识
- **model：** 使用的模型
- **choices：** 生成结果数组
- **content：** AI回复内容
- **usage：** Token使用量
- **finish_reason：** 结束原因

API总结

API作为积木块

API是现代软件开发的"乐高积木"，通过组合不同的 API，可以快速构建强大的应用。



综合练习：获取天气推荐路线

03

使用Agentic IDE开发C/S应用

Plan模式：让AI帮你规范开发流程

架构规划

接口契约

数据模型

部署说明

为什么建议使用Plan模式？

直接开发的痛点

开发C/S架构需要处理**职责分离、API约定、数据格式、错误处理**等复杂问题。如果直接让AI"帮我写个系统"，它可能会：

- 混淆前后端职责边界
- 缺少接口约束和规范
- 数据模型前后不一致
- 代码质量难以保证

Plan模式的好处

1. **提前规划架构** — 明确技术栈、文件结构、部署方式
2. **定义接口契约** — 约定API路径、参数、响应格式
3. **规范数据模型** — 统一字段类型、验证规则
4. **控制代码质量** — 基于文档生成，非零散记忆

Plan模式的核心思想

在写代码之前，**先让AI生成完整的设计文档**，明确架构、接口、数据模型等关键要素。基于文档生成代码，而非零散记忆，确保系统的一致性和可维护性。

Plan 文档关键要素1：高层架构设计

明确技术选型和项目组织方式

技术栈

服务端

> Node.js 18+ + Express 4.x

客户端

> 原生 HTML5 + CSS3 + ES6+

文件目录结构

```
project/
├── server/# 服务端代码
└── client/# 客户端代码
```

清晰的目录分离，便于团队协作和维护

部署方式

1. **本地启动**
   Node.js 服务器
2. **端口配置**
   监听端口 3000
3. **快速验证**
   便于开发和调试

Plan文档关键要素2：接口规范

统一响应格式

```
{
  "success": true,
  "data": { ... },
  "message": "操作成功"
}
```

定义接口列表

功能	方法	路径	参数
获取所有	GET	/api/spots	无
创建景点	POST	/api/spots	{name, city...}
删除景点	DELETE	/api/spots/:id	无

接口契约的价值

前后端协作基础
明确双方职责和数据格式

并行开发
前后端可同时开发，互不阻塞

接口文档
自动生成API文档供团队参考

注意事项

- 接口版本管理
- 错误处理规范
- 认证授权机制

Plan文档关键要素3：数据模型

什么是数据模型？

数据模型是**数据库的设计规范**，统一前后端数据结构。它定义了数据的字段名、类型、约束条件，确保数据在传输和存储过程中保持一致性。

景点数据模型示例

字段名	数据类型	约束	说明
id	VARCHAR(10)	PRIMARY KEY	唯一标识
name	VARCHAR(50)	NOT NULL	景点名称
rating	DECIMAL	1.0-5.0	评分
price	DECIMAL	>= 0	票价

数据模型的价值

统一数据结构

前后端使用相同的数据格式

数据验证

约束条件确保数据质量

便于维护

修改字段时影响范围清晰

设计建议

- 字段命名规范（小写下划线）
- 选择合适的数据类型
- 设置合理的约束条件
- 预留扩展字段

Plan文档关键要素3：数据模型

什么是数据模型？

可以用Excel中的Sheet的结构做类比

	A	B	C	D	E	F	G	H	I	J
1	ID	景点名称	城市	标签	评分	票价 (元)	开放时间	建议游玩	景点描述	图片路径
2	s1	外滩	Shanghai	地标、城市景观	4.8	0	全天	90	上海最具代表性的地标，可欣赏黄浦江两岸风光	img/s1_waitan.jpg
3	s2	东方明珠	Shanghai	地标、城市景观	4.7	199	09:00-21:00	120	上海标志性建筑之一，可登塔俯瞰城市全景。	img/s2_oriental-pearl-tower.jpg
4	s3	上海中心大厦	Shanghai	地标、城市景观	4.7	180	08:30-22:00	120	中国最高建筑之一，拥有世界级观景台。	img/s3_shanghai-tower.jpg
5	s4	金茂大厦	Shanghai	地标	4.6	120	09:00-21:00	90	经典摩天大楼，可登高俯瞰陆家嘴。	img/s4_jinmao-tower.jpg
6	s5	豫园	Shanghai	文化、古典园林	4.6	40	08:30-17:00	120	明代古典园林，江南园林代表。	img/s5_yuyuan.jpg
7	s6	城隍庙	Shanghai	文化	4.5	10	08:00-16:30	60	上海著名道教庙宇与传统商业区。	img/s6_chenghuang-miao.jpg
8	s7	龙华寺	Shanghai	文化	4.5	10	07:00-16:30	60	上海历史最悠久的佛教寺庙之一。	img/s7_longhua-temple.jpg
9	s8	静安寺	Shanghai	文化	4.6	50	07:30-17:00	60	市中心著名寺庙，香火旺盛。	img/s8_jingan-temple.jpg
10	s9	上海博物馆	Shanghai	博物馆	4.8	0	09:00-17:00	120	中国顶级博物馆之一，馆藏丰富。	img/s9_shanghai-museum.jpg
11	s10	上海科技馆	Shanghai	博物馆、亲子	4.7	60	09:00-17:15	180	适合亲子互动的科技类展馆。	img/s10_science-museum.jpg
12	s11	中国航海博物馆	Shanghai	博物馆	4.6	30	09:00-16:00	120	展示航海历史与文化。	img/s11_maritime-museum.jpg
13	s12	上海自然博物馆	Shanghai	博物馆、亲子	4.8	30	09:00-17:00	150	大型自然科学博物馆，适合儿童。	img/s12_natural-history-museum.jpg
14	s13	南京路步行街	Shanghai	购物、城市景观	4.6	0	全天	120	中国最繁华的商业街之一。	img/s13_nanjing-road.jpg
15	s14	淮海中路	Shanghai	购物	4.6	0	全天	120	高端购物街区。	img/s14_huaihai-road.jpg
16	s15	新天地	Shanghai	购物、美食	4.7	0	全天	120	融合历史与现代的商业区。	img/s15_xintiandi.jpg
17	s16	环球港	Shanghai	购物	4.5	0	10:00-22:00	120	大型购物中心。	img/s16_global-harbor.jpg
18	s17	迪士尼乐园	Shanghai	主题乐园、亲子	4.8	399	08:30-21:30	480	中国大陆首座迪士尼乐园。	img/s17_disneyland.jpg
19	s18	海昌海洋公园	Shanghai	主题乐园、亲子	4.6	299	09:00-20:00	300	大型海洋主题乐园。	img/s18_haichang-ocean-park.jpg
20	s19	欢乐谷	Shanghai	主题乐园	4.5	230	09:00-21:00	300	刺激娱乐项目丰富。	img/s19_happy-valley.jpg
21	s20	锦江乐园	Shanghai	主题乐园	4.3	120	09:00-17:00	180	老牌游乐园。	img/s20_jinjiang-park.jpg
22	s21	世纪公园	Shanghai	自然、公园	4.6	10	05:00-18:00	120	上海最大城市公园。	img/s21_century-park.jpg
23	s22	顾村公园	Shanghai	自然、公园	4.5	20	06:00-18:00	120	赏樱胜地。	img/s22_gucun-park.jpg
24	s23	共青森林公园	Shanghai	自然、公园	4.6	15	05:00-17:00	120	森林风格公园。	img/s23_gongqing-forest-park.jpg
25	s24	滨江大道	Shanghai	自然、城市景观	4.7	0	全天	90	沿江步道，适合散步。	img/s24_binjiang-avenue.jpg
26	s25	田子坊	Shanghai	文化、美食	4.5	0	10:00-22:00	90	文艺小资街区。	img/s25_tianzifang.jpg
27	s26	武康路	Shanghai	文化、城市景观	4.6	0	全天	90	历史建筑街区。	img/s26_wukang-road.jpg
28	s27	1933老场坊	Shanghai	文化	4.5	0	10:00-22:00	60	工业风建筑改造景点。	img/s27_1933-laochangfang.jpg
29										

Plan文档关键要素4：部署说明

开发环境启动步骤

1 安装依赖

```
cd server && npm install
```

2 启动服务器

```
npm start # 监听端口 3000
```

3 访问客户端

使用本地静态服务器托管 client/ 目录

推荐工具

http-server (Node.js)

命令行静态服务器

Python SimpleHTTPServer

Python内置静态服务器

核心目标

确保项目**可运行、可验证**。清晰的部署说明让任何人都能快速启动项目，降低上手门槛。

常见问题

- 端口被占用
- 依赖安装失败
- CORS跨域问题
- 路径配置错误

04

Demo 演示

实际案例展示



架构规划



接口契约



数据模型



部署说明

05

注意事项/ 避坑指南

新手开发C/S应用常见问题与解决方案

部署脚本　　CORS跨域　　错误处理

避坑指南1：安装部署脚本

⚠新手最容易遇到的问题

新手最容易卡在**环境配置和启动项目**。Node.js版本不对、依赖安装失败、端口被占用...各种问题让人崩溃。

常见痛点：

- 不知道需要安装哪些依赖
- 启动命令记不住
- 环境变量配置错误
- 服务端和前端启动顺序混乱

核心策略

把部署工作也交给AI！让AI生成完整的部署脚本和说明文档，你只需要复制粘贴执行即可。

"请生成start.sh，自动检查环境、安装依赖、启动服务端和前端。"

解决方案

1 要求生成"一键启动脚本"

"请生成start.sh，自动检查环境、安装依赖、启动服务端和前端。"

2 直接把错误日志喂给AI

遇到红色的终端报错，不需要自己读懂。直接复制给AI：

"运行报错了，日志如下，请帮我修复。"

预期效果

- 一条命令启动整个项目
- 自动检测和修复常见问题
- 清晰的错误提示和解决方案

避坑指南2：跨域问题（CORS）

什么是跨域？

跨域是**浏览器的安全机制**。不同端口（如前端5500，后端3000）被视为"不同源"，互相请求会被浏览器拦截。

典型场景：

- 前端运行在 http://localhost:5500
- 后端运行在 http://localhost:3000
- 浏览器认为它们是"不同源"
- 请求被拦截，控制台报错

解决方案

推荐：开启CORS

明确要求AI在服务端配置并开启CORS：

"必须在服务端配置并开启CORS，允许所有跨域请求。"

替代：同源部署

让AI生成由后端统一托管前端页面的代码，实现同源。

Express CORS配置示例

```
// 安装cors包
npm install cors
// 在server.js中配置
const cors = require('cors');
app.use(cors()); // 允许所有跨域请求
// 或者配置特定来源
app.use(cors({
  origin: 'http://localhost:5500'
}));
```

常见错误提示

```
Access to fetch at
'http://localhost:3000/api/spots' from origin
'http://localhost:5500' has been blocked by CORS
policy...
```

最佳实践

- 开发环境：允许所有来源
- 生产环境：配置白名单

避坑指南3：错误信息的输出

痛点描述

点按钮没反应，以为代码坏了。实际上错误只打印在控制台（Console）里，用户完全看不到任何提示。

典型场景：

- 用户点击提交按钮，页面没有任何反应
- 开发者打开控制台，发现红色错误信息
- 用户困惑，不知道发生了什么
- 体验极差，用户可能直接放弃使用

解决方案

强制要求"可见的错误提示"！前端页面在发起所有API请求时，必须做好错误处理。

"前端页面在发起所有API请求时，必须做好错误处理。如果失败，必须通过弹窗（Alert/Toast）将错误信息清晰展示给用户，绝对不能只打印在控制台。"

错误处理代码示例

```
fetch('/api/spots')
  .then(response => {
    if (!response.ok) {
      throw new Error('请求失败: ' + response.status);
    }
    return response.json();
  })
  .then(data => {
    // 处理成功响应
  })
  .catch(error => {
    //    不要只打印到控制台
    // console.error(error);

    //    显示用户可见的错误提示
    alert('操作失败: ' + error.message);
    // 或使用更优雅的Toast通知
    showToast('操作失败: ' + error.message, 'error');
  });
```

额外价值

这不仅提升用户体验，更是你**Vibe Coding过程中的排错利器**。清晰的错误提示能帮助你快速定位问题所在。

最佳实践

- 使用Toast替代Alert，体验更好
- 区分用户错误和系统错误
- 提供解决方案或重试按钮

避坑指南3：获取错误信息



总结

Context Engineering & Plan-Driven Development

掌握 C/S 架构开发

开启 AI 辅助编程之旅



架构设计

清晰的职责分离



接口规范

明确的协作契约



避坑指南

高效的开发实践



让 AI 成为你的编程伙伴

End

2.氛围编程的关键实践 (4)

- Agentic IDE基于Plan模式开发 C/S 应用